

NovaDrive CarLink — Developer Handover Guide

Target audience: Incoming developer taking over full ownership of the project. This document covers every layer of the system: database, backend API routes, frontend components, external integrations, environment setup, and operational gotchas.

1. Project Overview

NovaDrive (CarLink) is a luxury car rental and car leasing platform for the Kenyan market. Customers browse a vehicle fleet, reserve a car, sign a digital rental agreement, and pay via M-Pesa or credit card — all within a single web app.

Key user journeys:

Journey	Relevant pages
Browse & book a car	/ , /cars , BookingModal
Review & edit a reservation	/my-bookings
Sign the rental agreement	/contract?booking=<id>
Pay for the rental	/checkout?booking=<id>
Submit a car for leasing	/lease
Admin management	/admin

2. Tech Stack

Layer	Technology
Framework	Next.js 15+ (App Router)
UI	React 19, TypeScript, Tailwind CSS 3, Framer Motion, Lucide Icons
Database & Auth	Supabase (PostgreSQL, RLS, SSR cookies, Storage)

Layer	Technology
Digital Signatures	BoldSign API + official <code>boldsign</code> Node SDK
Payments	Paystack REST API (KES / M-Pesa & cards)
Maps & Geocoding	OpenStreetMap Nominatim API + Leaflet (client-only)
Email	Brevo (formerly SendGrid) via <code>BREVO_API_KEY</code>
Hosting	Netlify (primary) — <code>netlify.toml</code> + <code>@netlify/plugin-nextjs</code>

3. Environment Variables

Copy `.env.example` → `.env.local` and fill in every value before running.

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=      # Used by admin/webhook routes to bypass RLS

# Paystack
PAYSTACK_SECRET_KEY=            # sk_live_... or sk_test_...

# App
NEXT_PUBLIC_SITE_URL=http://localhost:3000

# BoldSign
BOLDSIGN_API_KEY=                # From https://app.boldsign.com → Settings → API Keys
BOLDSIGN_TEMPLATE_ID=            # Template ID of your rental agreement template
BOLDSIGN_WEBHOOK_SECRET=         # Signing secret from your BoldSign webhook config

# Brevo (email)
BREVO_API_KEY=
SUPPORT_EMAIL=no-reply@yoursite.com
SUPPORT_NAME=NovaDrive Support

# Cron job security
CRON_SECRET=                    # Random secret; must match Authorization header
```

Missing `BREVO_API_KEY` ? Emails are skipped and logged to console — the app still works. **Missing `BOLDSIGN_WEBHOOK_SECRET` ?** Webhook signature verification is skipped (not recommended in production).

4. Getting Started

```
# 1. Install dependencies (also compiles the BoldSign SDK — see §9)
npm install

# 2. Copy and fill env file
cp .env.example .env.local

# 3. Run dev server
npm run dev
```

The app runs on `http://localhost:3000`.

5. Database Schema

All tables live in the Supabase `public` schema with Row Level Security (RLS) enabled. Migrations are in `supabase/migrations/` (run them in order via the Supabase Dashboard or CLI).

Migration history

File	What it does
<code>001_initial_schema.sql</code>	Creates <code>profiles</code> , <code>cars</code> , <code>bookings</code> tables + RLS policies + new-user trigger
<code>002_seed_cars.sql</code>	Seeds initial vehicle data
<code>003_lease_requests.sql</code>	Creates <code>lease_requests</code> table
<code>004_reviews.sql</code>	Creates <code>reviews</code> table
<code>005_car_category.sql</code>	Adds <code>category</code> column to <code>cars</code>
<code>006_contract_fields.sql</code>	Adds ID/document/emergency contact fields to <code>bookings</code>
<code>007_zoho_contract_fields.sql</code>	Adds <code>contract_signed</code> , <code>contract_signed_at</code> (Zoho era — fields kept)
<code>008_zoho_payments.sql</code>	Adds <code>payment_status</code> , <code>payment_link</code> , <code>signed_at</code> , <code>paid_at</code>
<code>009_update_booking_status_constraint.sql</code>	Expands allowed <code>status</code> values
<code>010_remove_zoho_fields.sql</code>	Removes Zoho-specific columns (project migrated to BoldSign)

Table: `profiles`

1:1 with `auth.users`. Created automatically via trigger `on_auth_user_created`.

Column	Type	Notes
<code>id</code>	uuid PK	FK → <code>auth.users.id</code>
<code>full_name</code>	text	
<code>role</code>	text	'user' (default) or 'admin'

To promote a user to admin:

```
UPDATE public.profiles SET role = 'admin' WHERE id = 'USER_UUID';
```

Table: `cars`

Column	Type	Notes
<code>id</code>	uuid PK	
<code>make</code> , <code>model</code> , <code>year</code>	text/int	
<code>slug</code>	text unique	URL-friendly identifier
<code>category</code>	text	<code>small_car</code> , <code>mid_sized_car</code> , <code>suv</code> , <code>luxury</code> , <code>corporate_group</code>
<code>price_per_day</code>	numeric	USD
<code>price_per_week</code>	numeric	USD; falls back to <code>price_per_day * 7</code>
<code>price_per_month</code>	numeric	USD; falls back to <code>price_per_day * 30</code>
<code>available</code>	boolean	Master on/off switch
<code>units_available</code>	int	How many bookable units exist (decremented on payment, restored by cron)
<code>seats</code> , <code>transmission</code> , <code>fuel_type</code>		
<code>images</code>	text[]	Array of public image URLs
<code>features</code>	text[]	Feature bullets shown on car detail page

Table: `bookings`

The central table. A booking is created at reservation time and evolves through the workflow.

Renter info (filled at booking time): `user_id`, `car_id`, `full_name`, `phone`, `email`

Logistics: `pickup_at`, `return_at`, `pickup_location`, `dropoff_location`, `destination`, `rental_duration` (human label e.g. "3 days"), `driving_mode` ("Self-driven" | "Chauffeured"), `special_requests`

Financials: `total_amount` (USD), `paystack_reference`, `payment_status`, `paid_at`

Contract/identity (filled on the Contract page): `id_number`, `emergency_contact_name`, `emergency_contact_phone`, `profile_photo_url`, `id_front_url`, `id_back_url` (Supabase Storage URLs), `contract_signed` (boolean), `contract_signed_at`, `boldsign_document_id`, `signed_pdf_path`

Unit management: `units_restored` (boolean) — set to `true` by the cron job after `return_at` passes.

Status lifecycle:

```
pending → (contract signed) → signed_pending_payment → paid / confirmed
                                         ↘ cancelled
```

Allowed status values (migration 009): `pending`, `confirmed`, `cancelled`, `paid`, `pending_signature`, `signed_pending_payment`

Table: `lease_requests`

Submitted by car owners who want to list their vehicle. Fields: `brand`, `model`, `year`, `mileage_km`, `image_url`, `image_urls[]`, `phone_numbers[]`, `user_full_name`, `user_email`, `lease_duration_months`, `status` (`new` → `reviewing` → `accepted` | `rejected`)

Table: `reviews`

Post-rental feedback. Fields: `booking_id`, `car_id`, `rating` (1–5), `comment`, `liked`, `disliked`, `complaints`.

6. Supabase Client Helpers (`src/lib/supabase/`)

File	Usage
<code>client.ts</code>	Browser-side client — calls <code>createBrowserClient</code> . Used in 'use client' components.
<code>server.ts</code>	Server-side SSR client — reads auth from cookies. Used in API routes and Server Components.
<code>admin.ts</code>	Service-role client — bypasses RLS. Used in webhooks and the cron route that need admin access. Never expose to the browser.

7. Pricing Logic (`src/lib/pricing.ts`)

`computeRentalTotal(car, pickup, returnDate)` implements a **tiered** pricing model:

```
< 7 days    → days × price_per_day
≥ 7 days    → floor(days/7) weeks × price_per_week + leftover days × price_per_day
≥ 30 days   → floor(days/30) months × price_per_month + leftover days × price_per_day
```

Chauffeured surcharge: multiply the base by `1.35` (35% extra). Applied in `BookingModal`, `MyBookingsClient` (edit modal), `CheckoutClient`, and the `PATCH /api/bookings/[id]` route.

8. Key Frontend Components

Global State (React Context Providers)

`src/components/providers/`

Provider	Hook	What it manages
<code>AppUIProvider</code>	<code>useAppUI()</code>	<code>bookingOpen</code> / <code>authOpen</code> modals + <code>preselectedCar</code>
<code>UserSessionProvider</code>	<code>useUserSession()</code>	Supabase session — <code>user</code> , <code>profile</code> , <code>loading</code>

Both are mounted in the root `layout.tsx`. Any component that needs auth state or wants to open the booking modal uses these hooks.

BookingModal (`src/components/booking/BookingModal.tsx`)

A full-screen sliding modal mounted globally. Opened by `useAppUI().openBooking(car?)`.

- Fetches all `available = true` cars from Supabase on open, with client-side fuzzy search.
- If `preselectedCar` is set (user clicked "Book" on a car card), the vehicle is pre-selected.
- All three `LocationPickerInput` fields (Pickup, Dropoff, Destination) are **required** for submission.
- Date pickers enforce `min = now` to prevent past-date selections.
- Calls `POST /api/bookings` on submit → redirects to `/contract?booking=<id>`.
- **Email verification is required:** unverified accounts get an error before submitting.

MyBookingsClient (`src/components/booking/MyBookingsClient.tsx`)

Server-rendered list of the user's bookings, passed as `initialBookings` prop.

Contains two inline modals:

- **EditModal** — available for `status === 'pending'` bookings only. PATCHes `/api/bookings/[id]` ; recalculates price server-side.
- **ReviewModal** — available for bookings where `return_at` has elapsed and no review exists yet. POSTs to `/api/reviews` .

ContractClient (`src/components/contract/ContractClient.tsx`)

The most complex frontend component. A multi-section form that gates progress sequentially:

1. **Renter Details** — `id_number` , `emergency_contact_name` , `emergency_contact_phone` (typed inputs).
2. **Document Uploads** — Selfie, ID Front, ID Back. All 3 are required. Uploaded to Supabase Storage bucket `contract-docs` at path `{bookingId}/{slot}.{ext}` .
3. **Terms & Conditions** — scrollable T&C text; acknowledgement checkbox required.
4. **BoldSign Signing** — "Open Contract Agreement" button becomes active once fields + ack are done. Calls `POST /api/contract/boldsign-create` , receives `embed_url` , opens an iframe modal.
5. BoldSign fires `postMessage` → `{ action: "onDocumentSigned" }` from origin `https://app-eu.boldsign.com` . The listener sets `contractSigned = true` and closes the iframe.
6. **Proceed to Payment** — active only when all 5 above are complete. Uploads docs concurrently, updates booking row, navigates to `/checkout?booking=<id>` .

CheckoutClient (`src/components/checkout/CheckoutClient.tsx`)

Shows a booking summary with USD → KES conversion (rate: **1 USD = 130 KES**).

- "Edit booking details" button (for `pending` status only) opens an inline edit form using `LocationPickerInput` .
- "Pay with Paystack" → `POST /api/paystack/initialize` → redirects to Paystack-hosted checkout.

LocationPickerInput (`src/components/ui/LocationPickerInput.tsx`)

Reusable location field used in BookingModal, MyBookingsClient EditModal, and CheckoutClient.

- Debounced Nominatim search (350 ms delay, min 2 chars).
- Preset quick-select chips: JKIA, Wilson Airport, Westlands, CBD, Gigiri.
- "Pin on Map" toggle dynamically loads `LocationMap` (Leaflet, `ssr: false`).
- Clicking the map performs reverse geocoding and fills the text field.

9. API Routes Reference

`POST /api/bookings`

Creates a new booking. Requires auth. Validates car availability (`available = true` , `units_available > 0`). Inserts row with `status = 'pending'` , returns `{ id }` .

PATCH /api/bookings/[id]

Updates an existing **pending** booking. Only the owner can edit. Recalculates `total_amount` and `rental_duration` server-side using `computeRentalTotal`. Chauffeured surcharge is reapplied if `driving_mode = 'Chauffeured'`.

POST /api/contract/boldsign-create

Full server-side BoldSign flow: 1. Validates `BOLDSIGN_API_KEY` + `BOLDSIGN_TEMPLATE_ID`. 2. Verifies auth and loads booking (ownership check). 3. Calls `TemplateApi.sendUsingTemplate()` — pre-fills `full_name`, `id_number`, `phone`, `emergency_contact_phone`, `vehicle` fields. Sets `disableEmails: true` (signing is done in-app). Embeds `metaData: { bookingId }` for webhook correlation. 4. Saves `boldsign_document_id` on the booking row. 5. Calls `DocumentApi.getEmbeddedSignLink(documentId, signerEmail)`. 6. Returns `{ embed_url }`.

Error handling: Axios errors from BoldSign are caught and their `response.data` surfaced in the JSON response so the frontend can display the exact API error message.

POST /api/contract/boldsign-webhook

Called by BoldSign when a document event fires. 1. Reads raw body (before JSON parsing) for HMAC-SHA256 verification using `BOLDSIGN_WEBHOOK_SECRET` and header `X-BoldSign-Signature`. 2. Filters to `eventType === "Completed"` only (all others return `{ received: true, action: "ignored" }`). 3. Recovers `bookingId` from `payload.data.metaData.bookingId`. 4. Downloads signed PDF via `DocumentApi.downloadDocument(documentId)` → uploads to Supabase Storage `contract-docs/{bookingId}/signed-contract.pdf`. 5. Updates booking: `contract_signed = true`, `contract_signed_at`, `signed_pdf_path`.

POST /api/paystack/initialize

1. Fetches booking and verifies ownership.
2. Converts `total_amount` (USD) → KES × 100 (Kobo): `Math.round(amount * 130 * 100)`.
3. Generates unique reference: `NV_{bookingId[:8]}_{Date.now()}`.
4. Calls `https://api.paystack.co/transaction/initialize`.
5. Saves `paystack_reference` to booking row.
6. Returns `{ authorization_url, reference }` — frontend redirects the browser to the URL.

POST /api/paystack/verify

Called from the `/checkout/success` page with the `reference` from the URL query param. 1. Calls `GET https://api.paystack.co/transaction/verify/{reference}`. 2. If `status === 'success'`, sets booking `status = 'paid'` and decrements `cars.units_available`. 3. Uses `createAdminClient()` to bypass RLS.

POST /api/paystack/webhook

Paystack's server-to-server callback for `charge.success` events. 1. Verifies HMAC-SHA512 signature using `PAYSTACK_SECRET_KEY` and header `x-paystack-signature`. 2. On success: sets `booking.status = 'confirmed'`, `payment.status = 'paid'`, `paid_at`, decrements `units_available` on the car. 3. Calls `revalidatePath('/my-bookings')` to bust the Next.js cache.

Note: Both `/api/paystack/verify` (client-initiated) and `/api/paystack/webhook` (server-initiated) handle payment confirmation. Both decrement `units_available`, but verify guards against double-decrement with `existingBooking.status !== 'paid'` check.

GET /api/cron/restore-units

Restores car inventory after a booking's return date has passed. - Protected by `Authorization: Bearer <CRON_SECRET>` header. - Finds `status = 'paid'` bookings where `return_at < now` AND `units_restored = false`. - Increments `cars.units_available` by 1 per booking, then sets `units_restored = true` to prevent re-running. - Triggered daily at **03:00 UTC** via Netlify Scheduled Function (`netlify/functions/cron-restore-units.mts`).

POST /api/reviews

Creates a review for a completed booking. Standard auth + ownership validation.

Admin routes (/api/admin/...)

Admin-only endpoints for fleet and booking management. All validate `profile.role === 'admin'` server-side.

10. Route Protection & Middleware

`src/middleware.ts` runs on every request (excluding static assets).

- For `/admin*` routes: checks Supabase session cookie; if no session → redirect to `/` with `?auth=required`. If session exists but `profile.role !== 'admin'` → redirect to `/` with `?denied=admin`.
- For all other routes: middleware refreshes auth cookies (keeps sessions alive) and passes through.

11. Email Notifications (`src/lib/mailer.ts`)

Uses **Brevo** (formerly Sendinblue) via their REST API (`https://api.brevo.com/v3/smtp/email`).

Exported helpers: - `sendMail({ to, subject, html })` — core send function. - `paymentRequestHtml(...)` — email template sent after contract signing, prompting payment. - `confirmationHtml(...)` — booking confirmation email after payment succeeds.

If `BREVO_API_KEY` is not set, emails are **not sent** — the content is logged to console instead (safe for local development).

Also configure `SUPPORT_EMAIL` and `SUPPORT_NAME` env vars for the sender identity.

12. BoldSign Integration — Important Notes

Region

All SDK calls use `https://api-eu.boldsign.com`. If your BoldSign account is on the **US region**, change this to `https://api.boldsign.com` in both `boldsign-create/route.ts` and `boldsign-webhook/route.ts`.

`postMessage` `origin`

`ContractClient.tsx` listens for `event.origin === "https://app-eu.boldsign.com"`. Update this if your account is on the US region (`https://app.boldsign.com`).

Template fields

Only a subset of fields are currently pre-filled (lines 111–138 of `boldsign-create/route.ts`). Several fields are commented out. To enable additional pre-fill fields on the BoldSign template side, uncomment the relevant `field(...)` calls and ensure the field IDs match what's defined in your BoldSign template.

Postinstall script (CRITICAL)

The `boldsign` npm package does not always ship pre-compiled JS. `package.json` includes:

```
"postinstall": "node node_modules/typescript/lib/tsc.js -p node_modules/boldsign/tsconfig.json"
```

This compiles the SDK after `npm install`. **Do not remove this script.** It must run in your CI/CD environment as well (Netlify executes `postinstall` automatically).

13. Car Inventory Management

`cars.units_available` tracks how many units of each car model can be booked concurrently.

Event	Effect
New booking confirmed (payment success)	<code>units_available</code> decremented by 1 (webhook OR verify route)
Booking return date passes	<code>units_available</code> restored by 1 (cron job, daily 03:00 UTC)
Admin manually sets <code>available = false</code>	Car hidden from booking search entirely

The cron job guard (`units_restored` flag) prevents double-restoration if it runs multiple times.

14. Supabase Storage Buckets

Bucket	Contents	Access
<code>car-images</code>	Vehicle photos uploaded by admins	Public
<code>contract-docs</code>	Customer KYC documents + signed PDFs	Private (admin-only via service role)

Within `contract-docs` , the folder structure is:

```
contract-docs/  
  {bookingId}/  
    profile-photo.{ext}      ← selfie  
    document-front.{ext}     ← ID front  
    document-back.{ext}      ← ID back  
    signed-contract.pdf      ← final BoldSign PDF (written by webhook)
```

15. Deployment (Netlify)

`netlify.toml` configures: - Build command: `npm run build` - Publish dir: `.next` - Plugin: `@netlify/plugin-nextjs` (handles SSR) - Scheduled function: `cron-restore-units` runs daily at 03:00 UTC

The scheduled function at `netlify/functions/cron-restore-units.mts` calls `GET /api/cron/restore-units` with the `CRON_SECRET` header.

Required env vars in Netlify dashboard — all vars from `.env.local` must be added in `Site settings` → `Environment variables` , including `CRON_SECRET` .

Webhook URLs to configure in external dashboards:

Service	Setting	URL
BoldSign	Webhooks → your endpoint	<code>https://your-domain.com/api/contract/boldsign-webhook</code>
Paystack	Settings → Webhooks	<code>https://your-domain.com/api/paystack/webhook</code>

16. Directory Structure

```
Nova/  
├─ netlify/  
└─ functions/
```

```

└─┬─ cron-restore-units.mts # Netlify scheduled function (triggers cron route)
└─┬─ supabase/
└─┬─ migrations/ # SQL migration files 001-010
└─┬─ src/
└─┬─┬─ app/
└─┬─┬─┬─ page.tsx # Landing page (Hero, Fleet, About, Contact sections)
└─┬─┬─┬─ layout.tsx # Root layout – mounts providers + Navbar + Footer
└─┬─┬─┬─ cars/ # Fleet listing + [slug] car detail page
└─┬─┬─┬─ checkout/ # Checkout page + /success callback page
└─┬─┬─┬─ contract/ # Contract signing page
└─┬─┬─┬─ lease/ # Car owner onboarding page
└─┬─┬─┬─ my-bookings/ # Customer booking history + [id] detail page
└─┬─┬─┬─ my-leases/ # Customer lease request history
└─┬─┬─┬─ auth/ # Auth callback route (Supabase email confirm)
└─┬─┬─┬─ admin/ # Admin dashboard (fleet, bookings, reviews, leases, messages)
└─┬─┬─┬─┬─ api/
└─┬─┬─┬─┬─┬─ bookings/ # POST (create) + [id]/PATCH (edit)
└─┬─┬─┬─┬─┬─ contract/
└─┬─┬─┬─┬─┬─┬─ boldsign-create/ # Generate BoldSign embed URL
└─┬─┬─┬─┬─┬─┬─┬─ boldsign-webhook/ # Receive BoldSign completion events
└─┬─┬─┬─┬─┬─┬─ paystack/
└─┬─┬─┬─┬─┬─┬─┬─┬─ initialize/ # Start a Paystack transaction
└─┬─┬─┬─┬─┬─┬─┬─┬─ verify/ # Client-side verification after redirect
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ webhook/ # Server-to-server Paystack event handler
└─┬─┬─┬─┬─┬─┬─┬─┬─ cron/
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ restore-units/ # Restores car units after rental expiry
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ reviews/ # POST a new review
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ lease/ # Lease request CRUD
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ admin/ # Admin-only booking/car management endpoints
└─┬─┬─┬─┬─┬─ components/
└─┬─┬─┬─┬─┬─┬─ providers/
└─┬─┬─┬─┬─┬─┬─┬─ app-ui-provider.tsx # Global modal state (booking, auth)
└─┬─┬─┬─┬─┬─┬─┬─┬─ user-session-provider.tsx # Supabase session context
└─┬─┬─┬─┬─┬─┬─ booking/
└─┬─┬─┬─┬─┬─┬─┬─┬─ BookingModal.tsx # Full-screen booking form (global modal)
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ MyBookingsClient.tsx # Booking list + EditModal + ReviewModal
└─┬─┬─┬─┬─┬─┬─┬─┬─ checkout/
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ CheckoutClient.tsx # Checkout summary + edit form + pay button
└─┬─┬─┬─┬─┬─┬─┬─┬─ contract/
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ ContractClient.tsx # Contract signing wizard (BoldSign iframe)
└─┬─┬─┬─┬─┬─┬─┬─┬─ ui/
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ LocationPickerInput.tsx # Geocoding input with Leaflet map toggle
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ LocationMap.tsx # Leaflet map (dynamic import, no SSR)
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ admin/ # Admin tab components (Fleet, Bookings, Reviews, Leases, Messages)
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ landing/ # Hero, FeaturedFleet, About, Services, Offers, Contact sections
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─ lib/
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ pricing.ts # computeRentalTotal – tiered day/week/month pricing
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ paystack.ts # Paystack helper (used in some older routes)
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ mailer.ts # Brevo email sender + HTML templates
└─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─┬─ supabase/

```

```

|   |   |   | client.ts           # Browser Supabase client
|   |   |   | server.ts          # SSR Supabase client (reads cookies)
|   |   |   | admin.ts           # Service-role client (bypasses RLS)
|   |   |   | middleware.ts       # Admin route protection
|   |   |   | types/
|   |   |   |   | database.ts     # TypeScript types for all DB tables

```

17. Common Gotchas & Things to Watch

1. **Chauffeured pricing is 1.35× the base** — applied in 4 places: `BookingModal` , `MyBookingsClient` (edit modal), `CheckoutClient` , and `PATCH /api/bookings/[id]` . If the rate changes, update all four.
2. **USD → KES conversion is hardcoded at 130** in `CheckoutClient.tsx` and `api/paystack/initialize/route.ts` . For live rates, replace with an FX API call.
3. **units_available can be decremented twice** if both the Paystack webhook and the verify endpoint fire on the same payment. The verify route guards against this with `existingBooking.status !== 'paid'` , but the webhook does not. Monitor for inventory drift in production.
4. **BoldSign embed URL is short-lived** (typically 10 minutes). If a user sits on the contract page too long before clicking "Open Contract Agreement", the API call will succeed but the iframe may refuse to load. A page refresh will generate a new URL.
5. **Email verification is required to book** — enforced in `BookingModal.submit()` via `user.email_confirmed_at` . Users who sign up but don't verify their email can browse but cannot submit a booking.
6. **Leaflet requires no SSR** — `LocationMap` is always dynamically imported with `{ ssr: false }` . If you add it to any new component, do the same or the Next.js build will fail with a `window is not defined` error.
7. **Bookings can only be edited when status === 'pending'** — enforced both in the UI (`MyBookingsClient` , `CheckoutClient`) and server-side in `PATCH /api/bookings/[id]` .
8. **SUPABASE_SERVICE_ROLE_KEY must never be exposed to the browser** — it's only used in `admin.ts` , which is only imported by server-only routes and the cron endpoint.

Document last updated: August 2026